

Descripción.

Este programa utiliza un modelo de **árbol de procesos** mediante llamadas a `fork()`. El proceso padre actúa como cliente, creando un primer hijo que funciona como tenedor y un segundo hijo (servidor). La comunicación entre procesos se realiza mediante **pipes con nombre (FIFOs)**, permitiendo la transferencia de solicitudes y respuestas entre cliente, tenedor y servidor. Este modelo permite la simulación de un servidor que recibe solicitudes, las procesa y responde al cliente.

Modelo de creación de procesos hijos

Se usa `fork()` para crear procesos hijos:

```
pid_t pid_fork = fork();
if (pid_fork == 0) {
    pid_t pid_server = fork();
    if (pid_server == 0) {
        server(); // proceso servidor
    } else {
        fork(); // proceso tenedor
    }
} else {
    cliente(0); // proceso cliente
}
```

Explicación del flujo:

1. **Proceso padre** (main) crea un **hijo** con `fork()`.
 - `pid_fork == 0` → es el hijo.
 - `pid_fork != 0` → es el padre.
 2. **Hijo del primer fork:**
 - Crea otro hijo (`pid_server = fork()`) para el **servidor**.
 - El hijo original del primer fork actúa como **tenedor** (proxy entre cliente y servidor).
 3. **Padre original:**
 - Ejecuta el **cliente**, que envía solicitudes al tenedor, se espera 1 segundo a que arranquen el servidor y el fork.
-

Modelo de comunicación entre procesos

Se usan **pipes con nombre** (FIFOs) para la comunicación:

```
const char* client_to_fork = "/tmp/ctf";
const char* fork_to_server = "/tmp/fts";
const char* server_to_fork = "/tmp/stf";
const char* fork_to_client = "/tmp/ftc";
```

Flujo de datos:

Cliente ---> ctf ---> Tenedor ---> fts ---> Servidor
Servidor ---> stf ---> Tenedor ---> ftc ---> Cliente

- **Cliente → Tenedor:** El cliente escribe en `client_to_fork` (`/tmp/ctf`). El tenedor lee desde este FIFO.
- **Tenedor → Servidor:** El tenedor traduce la solicitud HTTP al protocolo TAKE y la escribe en `fork_to_server` (`/tmp/fts`). El servidor lee desde este FIFO.
- **Servidor → Tenedor:** El servidor procesa la solicitud y escribe la respuesta en `server_to_fork` (`/tmp/stf`). El tenedor lee la respuesta.
- **Tenedor → Cliente:** Finalmente, el tenedor escribe la respuesta HTTP en `fork_to_client` (`/tmp/ftc`). El cliente la lee y la imprime.

Resumen del modelo

Función / Proceso	Tipo de proceso	Canal	Descripción
<code>cliente()</code>	Padre	-	Crea todos los procesos y actúa como cliente.
<code>forc()</code> (primer hijo)	Hijo (tenedor)	FIFOs	Recibe solicitudes del cliente y las traduce al protocolo interno.
<code>server()</code> (segundo hijo)	Hijo (servidor)	FIFOs	Procesa comandos TAKE y envía la respuesta al tenedor.
Comunicación	FIFO (pipes con nombre)	Unidireccional	Cliente ↔ Tenedor ↔ Servidor ↔ Tenedor ↔ Cliente.